

CSN 342

Deep Learning

Course Instructor/Faculty: Dr. Vyomika Singh

UNIT 2: Feed Forward Networks (Topics covered in this PPT)

Multilayer Perceptron (MLP) and activation functions. **Logistic Regression**, Learning parameters, Loss functions, Empirical Risk Minimization, Gradient Descent and Backpropagation algorithm. **Regularization techniques. Difficulty of training deep neural networks, Greedy layer wise training.**

CSN 342

Deep Learning

Logistic Regression

Logistic regression is a common machine learning algorithm for binary classification and predicts a **binary categorical variable** through a logistic function.

The logistic regression model passes the outcome of a linear function of features through a logistic function to calculate the **probability of an occurrence**. The model then maps the probability to binary outcomes.

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-z}}$$

The **sigmoid function** maps any real-valued number $x \in \mathbb{R}$ to a value between 0 and 1. This ensures that output falls within a valid probability range.

Example applications:

1. Email spam detection (spam / not spam)
2. Disease prediction (disease / no disease)
3. Student pass/fail prediction

CSN 342

Deep Learning

Logistic Regression: sigmoid function

It is natural to wonder how a logistic regression model can ensure its output represents a probability, always **outputting a value between 0 and 1**. As it happens, there's a family of functions called logistic functions whose output has those same characteristics. The standard logistic function, also known as the sigmoid function (sigmoid means "s-shaped"), has the formula:

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

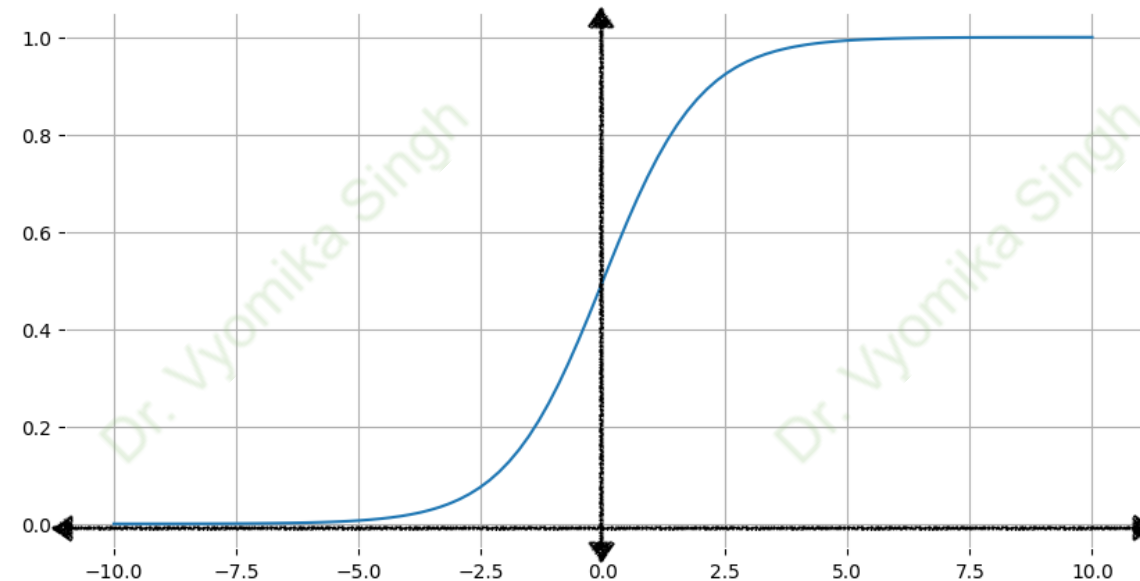
where:

sigmoid (x) or f(x) is the output of the sigmoid function.
e is Euler's number; a mathematical constant ≈ 2.71828 .
x is the input to the sigmoid function.

CSN 342

Deep Learning

Logistic Regression: sigmoid function



Properties of Sigmoid:

1. Output range: **(0, 1)**
2. If $z = +\infty$, output: **1**
3. If $z = -\infty$, output: **0**
4. If $z = 0$, output: **0.5**
5. Smooth and differentiable: good for gradient-based optimization

Interpretation:

$$P(y=1|x) = \sigma(wTx + b)$$

So sigmoid converts the linear score into a **probability of class 1**.

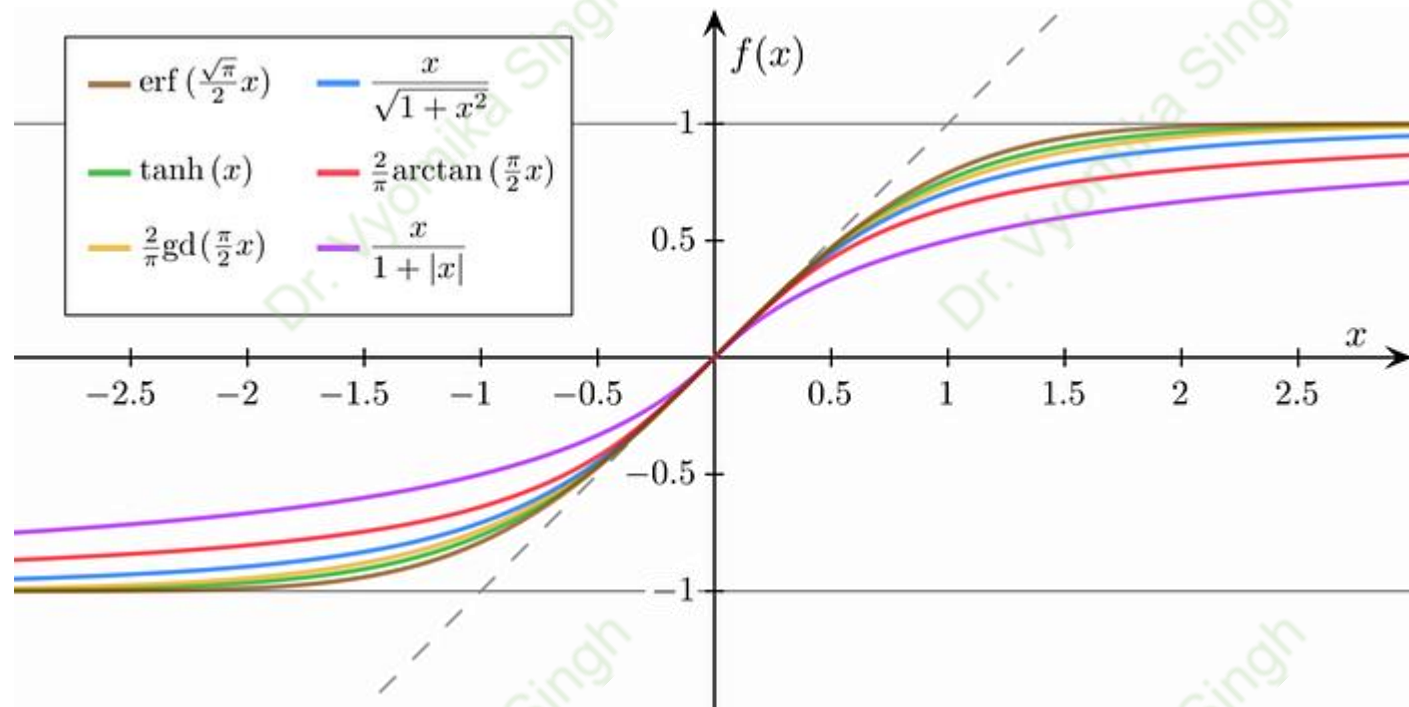
Figure: Graph of the sigmoid function. The curve approaches 0 as x values decrease to negative infinity, and 1 as x values increase toward infinity.

CSN 342

Deep Learning

Logistic Regression: sigmoid function

But what is so special about Sigmoid? For instance, the figure below shows six possible alternative functions.



Question: Of all possible choices, why Sigmoid? An infinite number of functions can monotonically map a real input (x) to the range $[0,1]$.

CSN 342

Deep Learning

Logistic Regression: sigmoid function

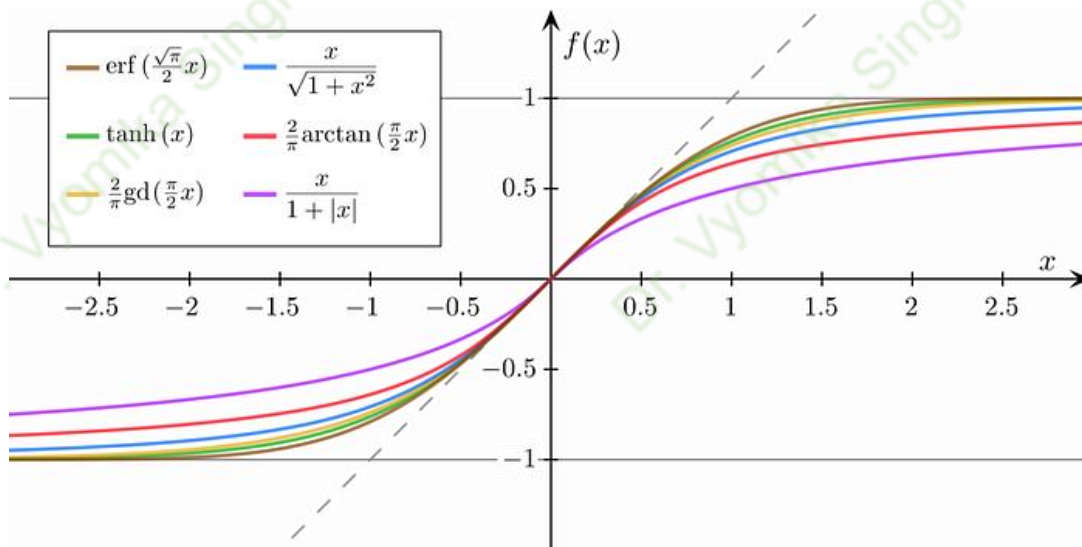
1. We Need Output in (0, 1) for Probability

- Logistic regression models **probability**: $P(y=1 | x)$

- Sigmoid outputs exactly in (0, 1)** which is perfect for probability.

- Many curves in the plot (like tanh, arctan, erf) output in **(-1, 1)** or other ranges; **not directly usable** as probability without extra scaling.

Sigmoid fits the **probability interpretation naturally**.



CSN 342

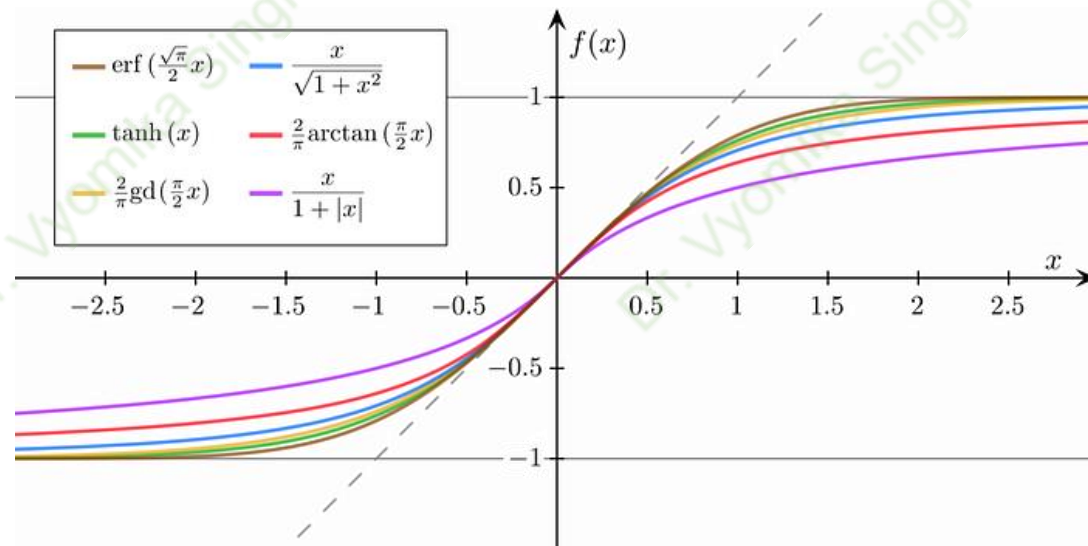
Deep Learning

Logistic Regression: sigmoid function

2. Direct Link to Log-Odds (Logit Model):
Sigmoid is not just a random choice, it is **mathematically the correct inverse of the logit**.

3. Works Perfectly with Cross-Entropy (Log Loss):

- Sigmoid + Binary Cross-Entropy gives a **convex loss function**.
- Convex loss - **single global minimum** - stable, reliable training.
- If you use tanh / arctan / others, you **lose this clean convex optimization property** for logistic regression.



CSN 342

Deep Learning

Logistic Regression: sigmoid function

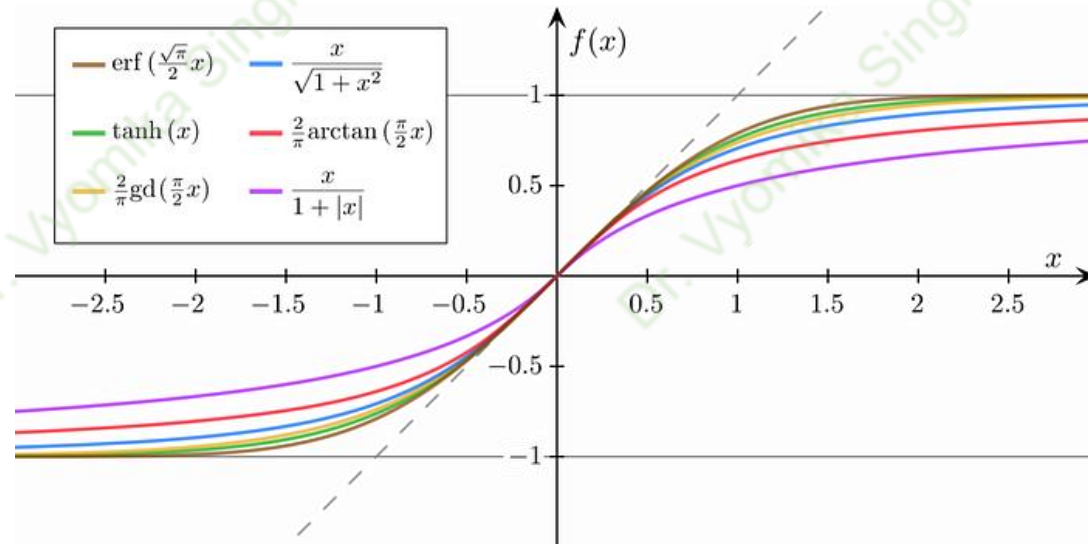
4. Smooth, Monotonic, and Differentiable (Like others, but...)

• Yes, many curves in the figure are also smooth and S-shaped.

• But sigmoid is:

- **Monotonic** (higher score $\rightarrow$ higher probability)
- **Simple derivative:**
- $\sigma'(z) = \sigma(z)(1-\sigma(z))$
- This makes gradient descent **simple and efficient.**

Others are usable, but **sigmoid** is **computationally simpler and cleaner.**



CSN 342

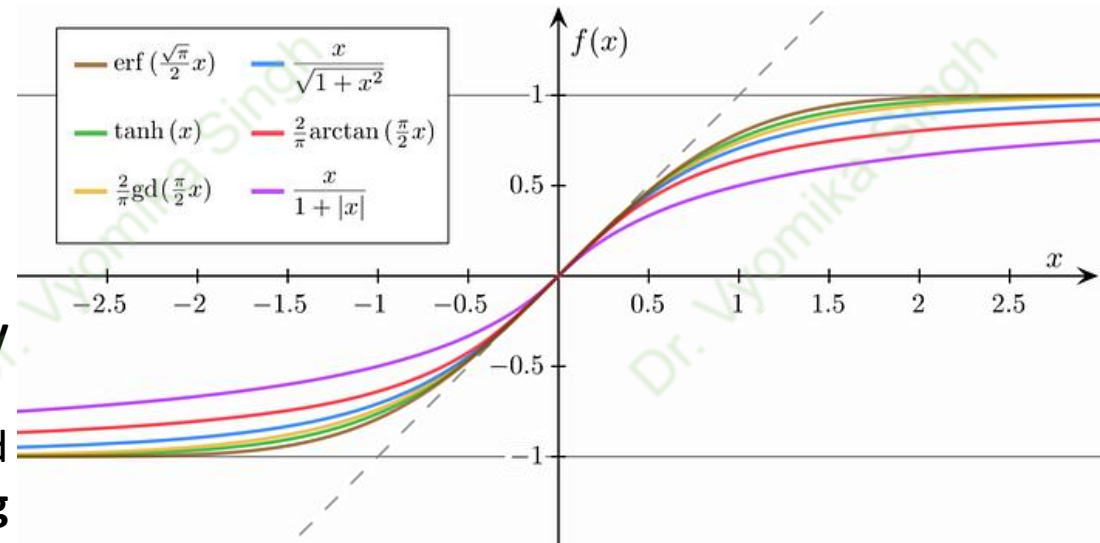
Deep Learning

Logistic Regression: sigmoid function

5. Natural Decision Boundary at 0.5

For sigmoid:

- $\sigma(0) = 0.5$
- Decision rule:
 - If $wTx + b \geq 0$ then class 1
 - Else, class 0
- This matches **linear decision boundary** perfectly.
- Other functions (like tanh) are centered at **0**, not **0.5**, so one needs **extra shifting and interpretation**.



Although many S-shaped functions exist, sigmoid is used in logistic regression because it naturally models probabilities via the inverse log-odds, works with cross-entropy to give a convex objective, and provides a simple, interpretable decision boundary at 0.5

CSN 342

Deep Learning

Logistic Regression: Cost Function

In logistic regression, the model output is:

1. $y^{\wedge} = \sigma(z) = 1 / (1 + e^{\text{power}(-z)})$, where $z = w^T x + b$;
2. $y^{\wedge}$ is **not** a raw number; it is a **probability**: $y^{\wedge} = P(y=1|x)$
3. The true label $y \in \{0,1\}$

If we use **Mean Squared Error (MSE)** here:

1. The optimization becomes **non-convex**,
2. Training can get stuck in **local minima**,
3. Convergence becomes **slow and unstable**.

Hence it is a need to use a cost function derived from **probability and maximum likelihood**:
Log Loss / Binary Cross-Entropy Loss

CSN 342

Deep Learning

Logistic Regression: Cost Function

Model:

$$\hat{y}_i = \sigma(w^T x_i + b)$$

Cost function:

$$J(w, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Goal:

$$\min_{w, b} J(w, b)$$

Logistic regression uses the **binary cross-entropy (log loss) cost function**, derived from maximum likelihood, which penalizes wrong confident predictions and gives a convex, stable optimization when combined with the sigmoid function.

CSN 342

Deep Learning

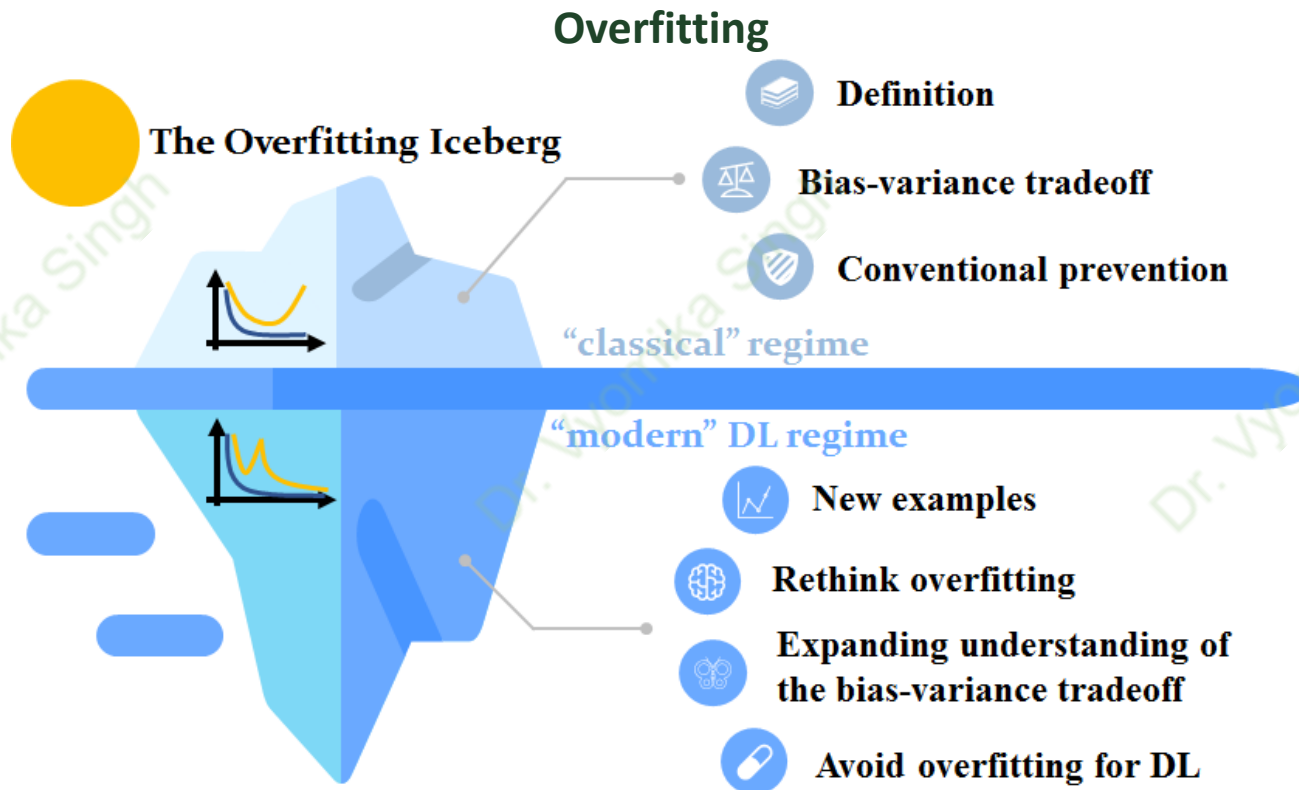


Fig: Modern ML practitioners witness phenomena that cast new insight on the bias-variance trade-off philosophy. The evidence that very complex neural networks also generalize well on test data motivates us to rethink overfitting; Source: ML @ CMU

CSN 342

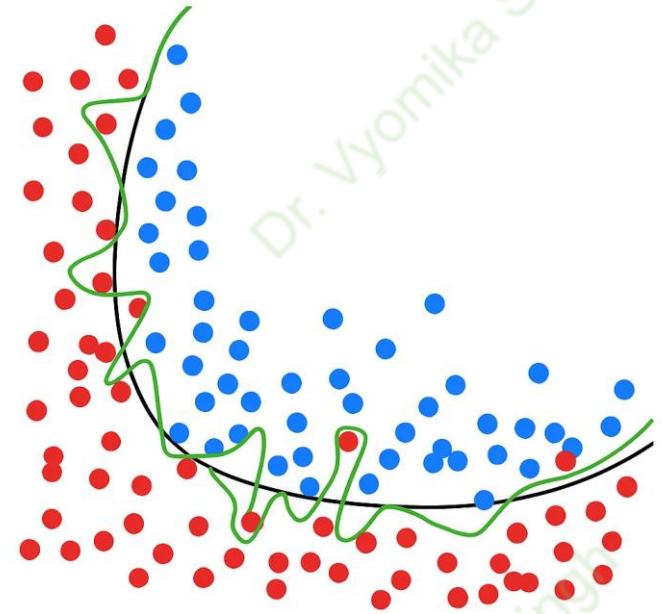
Deep Learning

Overfitting

Overfitting happens **when the model learns too much from the training data, including noise and outliers**. It performs very well on training data but poorly on test data.

Overfitting happens due to:

1. Model too complex
2. Too many features
3. Very little data
4. No regularization
5. High variance



CSN 342

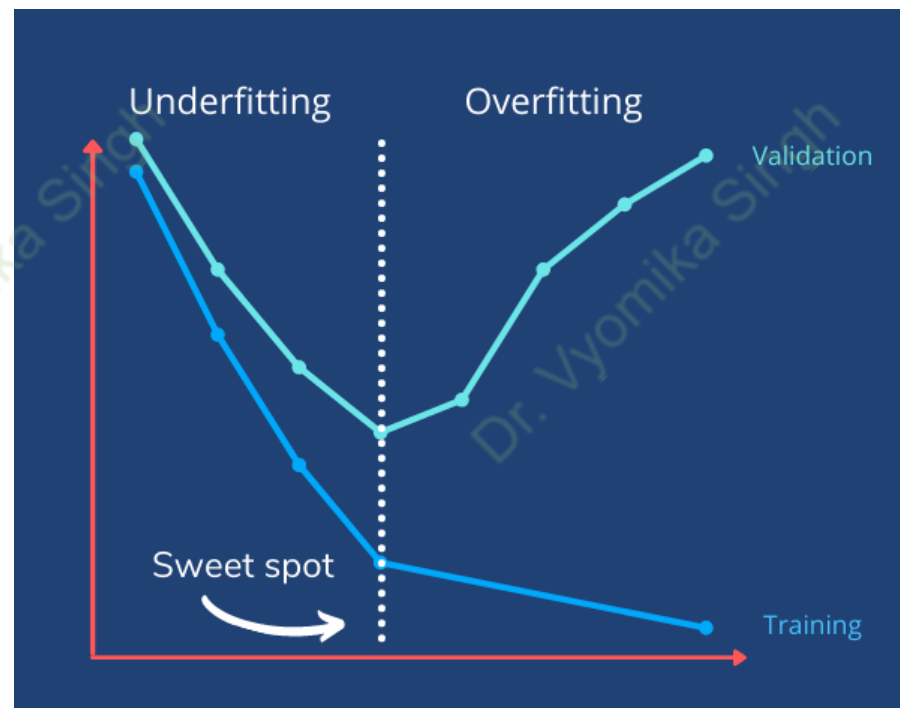
Deep Learning

Overfitting

Bias–Variance Inside Overfitting

Overfitting is mainly caused by high variance:

1. High variance means model reacts too strongly to training data
2. Learns noise as patterns
3. Low bias because the model is extremely flexible

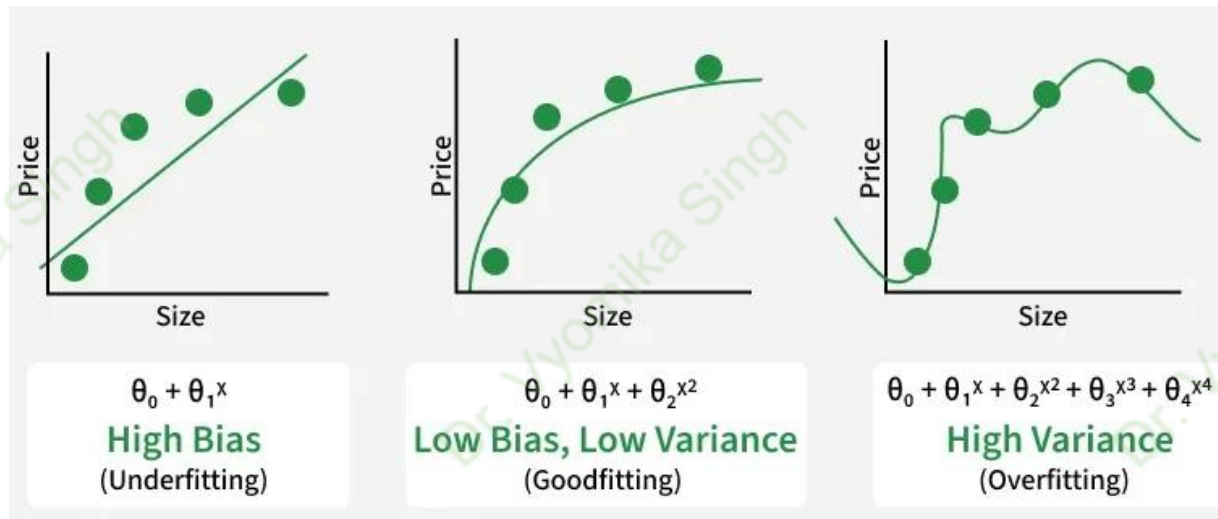


Overfitting = Low Bias + High Variance

CSN 342

Deep Learning

Underfitting, Overfitting and Good fitting



1. **Underfitting** : Straight line trying to fit a curved dataset but cannot capture the data's patterns, leading to poor performance on both training and test sets.
2. **Overfitting**: A squiggly curve passing through all training points, failing to generalize performing well on training data but poorly on test data.
3. **Appropriate Fitting**: Curve that follows the data trend without overcomplicating to capture the true patterns in the data.

CSN 342

Deep Learning

Overfitting

Various approaches have been proposed and applied in practice to avoid overfitting. They target different aspects of a machine learning task.

1. **Design of datasets:** In general, the model generalizes better with more training data. In addition to gathering more data, data augmentation is also widely adopted.
2. **Use of a simpler model:** As a larger function space is more prone to overfitting, a simpler model is usually preferred.
3. **Regularization in various forms:** Explicit regularization includes adding a penalty term, dropout for Deep Neural Networks (DNN), weight decay, etc. Implicit regularizations include early stopping and batch normalization, etc.

CSN 342

Deep Learning

Overfitting

4. **Ensembling:** Ensembling includes bagging, stacking, and boosting, etc. It has been shown that an ensemble of many simple models can fit the data with both low bias and low variance and thus prevent overfitting.
5. **Validation:** The main method of detecting overfitting in the first place is to leave part of the training data as a validation set (or a development set), and compare the model's performance between the training and validation sets. It will also allow one to measure how effective their overfitting prevention strategies are.

CSN 342

Deep Learning

Regularization: Concept

Goal: **Reduce Overfitting** usually achieved by reducing model capacity and/or reduction of the variance of the predictions

In the context of deep learning, **regularization can be understood as the process of adding information / changing the objective function to prevent overfitting**

Common regularization techniques for DNNs:

- Early stopping
- L1/L2 regularization (norm penalties)
- Dropout and many more..

CSN 342

Deep Learning

Regularization: Concept

Why do we need regularization?

Problem: Overfitting

Model learns training data too well (memorizes) and performs poorly on new/unseen data

Deep networks have millions of parameters. They can fit noise, not just signal.
Regularization = techniques to make the model generalize better

CSN 342

Deep Learning

Regularization: Concept

Regularization is implemented by modifying the loss function. In a typical machine learning model, the loss function measures how well the model's predictions match the actual data. Regularization adds an extra term to this loss function that penalizes large weights.

$$\text{Loss} = \text{Original Loss} + \lambda \times \text{Regularization Term}$$

Original Loss: Measures the difference between the model's predictions and the actual values.

Regularization Term: Adds a penalty for larger weights.

λ (lambda): A hyperparameter that controls the strength of the penalty. A larger λ means more regularization.

CSN 342

Deep Learning

Regularization: Concept

L1 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

L2 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M W_j^2$$

Loss function

Regularization
Term

CSN 342

Deep Learning

Regularization: Concept

- LASSO Regression:** A regression model which uses the L1 Regularization technique is called **LASSO (Least Absolute Shrinkage and Selection Operator)** regression. It adds the absolute value of magnitude of the coefficient as a penalty term to the loss function (L). This penalty can shrink some coefficients to zero which helps in selecting only the important features and ignoring the less important ones.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m |w_i|$$

Where

- m : Number of Features
- n : Number of Examples
- y_i : Actual Target Value
- $\hat{y}_i$: Predicted Target Value

CSN 342

Deep Learning

Regularization: Concept

2. **RIDGE Regression:** A regression model that uses the L2 regularization technique is called Ridge regression. It adds the squared magnitude of the coefficient as a penalty term to the loss function (L). **It handles multicollinearity by shrinking the coefficients of correlated features instead of eliminating them.**

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m w_i^2$$

Where,

- n : Number of examples or data points
- m : Number of features i.e predictor variables
- y_i : Actual target value for the i th example
- $\hat{y}_i$: Predicted target value for the i th example
- w_i : Coefficients of the features
- λ : Regularization parameter that controls the strength of regularization

CSN 342

Deep Learning

Regularization: Concept

3. ELASTIC NET: is a combination of both L1 as well as L2 regularization. That shows that we add the absolute norm of the weights as well as the squared measure of the weights. With the help of an extra hyperparameter that controls the ratio of the L1 and L2 regularization.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left((1 - \alpha) \sum_{i=1}^m |w_i| + \alpha \sum_{i=1}^m w_i^2 \right)$$

Where

- n : Number of examples (data points)
- m : Number of features (predictor variables)
- y_i : Actual target value for the i^{th} example
- $\hat{y}_i$: Predicted target value for the i^{th} example
- w_i : Coefficients of the features
- λ : Regularization parameter that controls the strength of regularization
- α : Mixing parameter where $0 \leq \alpha \leq 1$ and $\alpha = 1$ corresponds to Lasso (L_1) regularization, $\alpha = 0$ corresponds to Ridge (L_2) regularization and Values between 0 and 1 provide a balance of both L1 and L2 regularization

CSN 342

Deep Learning

Early Stopping

Concept:

Early stopping is a **regularization technique** where we **stop training** when performance on the **validation set** stops improving.

During training:

- Training loss usually **keeps decreasing**
- Validation loss **decreases first**, then may **start increasing** (overfitting starts)

Early stopping **stops training at the point of best validation performance**, not when training loss is minimum.

Why It Works:

- Prevents the model from **memorizing training data**
- Keeps the model at a **good bias–variance tradeoff**
- Acts like **implicit regularization** (limits effective model complexity)

CSN 342

Deep Learning

Dropout

Dropout is a **regularization technique** where, during training, we **randomly turn off (drop)** a fraction of neurons in each layer.

- If dropout rate = 0.5:
 - 50% of neurons are **randomly removed** in each iteration
- At test time:
 - **All neurons are used**, but outputs are scaled appropriately

Why It Works

- Prevents **co-adaptation** of neurons (neurons depending too much on each other)
- Forces the network to learn **robust, redundant features**
- Acts like training **many different smaller networks** and averaging them

CSN 342

Deep Learning

Batch Normalization (BatchNorm)

Batch Normalization **normalizes the activations** of a layer for each mini-batch so that they have:

- **Mean ≈ 0**
- **Variance ≈ 1**

Then it applies a **learnable scale and shift**.

Why It Works

- Reduces **internal covariate shift** (distribution of activations changing during training)
- Makes training:
 - **Faster**
 - **More stable**
- Allows:
 - Higher learning rates
 - Less sensitivity to initialization
- Also gives a **regularization effect** (slight noise from batch statistics)

CSN 342

Deep Learning

Comparison

Technique	Main Purpose	How it Helps
Early Stopping	Prevent overfitting	Stops training before memorization
Dropout	Regularization	Randomly removes neurons = robust features
Batch Normalization	Training stability + speed	Normalizes activations = faster, smoother training

CSN 342

Deep Learning

Greedy layer wise training

Greedy layer-wise training is a training strategy for deep neural networks where, we train the network one layer at a time, starting from the first layer, instead of training all layers together end-to-end.

1. Each layer is trained to learn useful representations of the input.
2. Once a layer is trained, its parameters are frozen.
3. Then the next layer is added and trained on top of the previous one.
4. Finally, the whole network may be fine-tuned using backpropagation.
5. It is called “greedy” because each layer is optimized locally without considering the global optimum of the full network.

https://proceedings.neurips.cc/paper_files/paper/2006/file/5da713a690c067105aeb2fae32403405-Paper.pdf

CSN 342

Deep Learning

Greedy layer wise training

Why Was It Needed? (Historical Motivation)

In early deep learning (pre-2012 era), training **very deep networks** with backpropagation was **difficult** because of:

1. **Vanishing gradients**
2. **Poor initialization**
3. **Getting stuck in bad local minima**

Networks failed to converge or learned very poor solutions. Greedy layer-wise training was introduced to:

- Provide **good initialization** for deep networks
- Make training **stable and feasible**
- Learn features **progressively from low-level to high-level**

https://proceedings.neurips.cc/paper_files/paper/2006/file/5da713a690c067105aeb2fae32403405-Paper.pdf

CSN 342

Deep Learning

Comparison

Aspect	Greedy Layer-Wise Training	Backpropagation
Training style	Layer by layer	End-to-end (all layers together)
Optimization	Local (each layer separately)	Global (whole network loss)
Weight updates	One layer at a time; previous layers frozen	All layers updated together
Use of labels	Often unsupervised pretraining + supervised fine-tuning	Fully supervised (usually)
Initialization	Learns good initial weights	Needs good initialization to work well
Stability in very deep nets	More stable (historically)	Can suffer from vanishing/exploding gradients
Complexity	More steps, slower	Simpler pipeline
Optimality	Not globally optimal (greedy)	Optimizes global objective
Modern usage	Rare (mostly historical / special cases)	Standard method today